

# TP3 — Administration & Automatisation

exploiter l'environnement Azure de **ShopEasy**

Inventorier, administrer, superviser et optimiser une infrastructure Azure avec **Azure CLI**, **Bash**, **Python** et **Azure Monitor**.

## Binôme

Louis SCARFONE

Maxence BOURRAGUÉ

## Cadre

Mastère Dev Manager Full Stack — RNCP niveau 7

Cas fil rouge : **ShopEasy**

Azure for Students — région swedencentral

**12**

ateliers + quiz

**4**

scripts CLI/Bash/Python

≈ **47 \$**

coût mensuel estimé

# Sommaire

## Table des matières

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Atelier 1 — Prise en main d’Azure CLI</b>                       | <b>3</b>  |
| 1.1      | Contexte — du déploiement à l’exploitation . . . . .               | 3         |
| 1.2      | Centralisation des variables — variables.sh . . . . .              | 3         |
| 1.3      | Version d’Azure CLI & subscription active . . . . .                | 4         |
| 1.4      | Groupe de ressources cible . . . . .                               | 4         |
| 1.5      | Configuration des valeurs par défaut . . . . .                     | 5         |
| 1.6      | Travail demandé . . . . .                                          | 5         |
| 1.7      | Capture . . . . .                                                  | 6         |
| <b>2</b> | <b>Atelier 2 — Inventorier les ressources Azure</b>                | <b>6</b>  |
| 2.1      | Lister les ressources . . . . .                                    | 7         |
| 2.2      | Exporter l’inventaire (JSON + TSV) . . . . .                       | 7         |
| 2.3      | Identifier les types . . . . .                                     | 7         |
| 2.4      | Tableau d’inventaire . . . . .                                     | 8         |
| 2.5      | Travail demandé . . . . .                                          | 8         |
| <b>3</b> | <b>Atelier 3 — Normaliser les tags d’exploitation</b>              | <b>9</b>  |
| 3.1      | Stratégie de tags d’exploitation . . . . .                         | 9         |
| 3.2      | État avant normalisation . . . . .                                 | 9         |
| 3.3      | Appliquer les tags (RG puis VM) . . . . .                          | 10        |
| 3.4      | Vérification et couverture . . . . .                               | 10        |
| 3.5      | Pourquoi les tags sont utiles (FinOps & gouvernance) . . . . .     | 11        |
| 3.6      | Captures . . . . .                                                 | 11        |
| <b>4</b> | <b>Atelier 4 — Administrer les machines virtuelles</b>             | <b>12</b> |
| 4.1      | Lister les VM et leur état . . . . .                               | 12        |
| 4.2      | Exécuter une commande distante (vérifier le serveur web) . . . . . | 13        |
| 4.3      | Redémarrer et désallouer . . . . .                                 | 13        |
| 4.4      | stop ≠ deallocate (point FinOps essentiel) . . . . .               | 14        |
| 4.5      | Tableau récapitulatif des VM . . . . .                             | 14        |
| 4.6      | Travail demandé . . . . .                                          | 14        |
| 4.7      | Capture . . . . .                                                  | 15        |
| <b>5</b> | <b>Atelier 5 — Construire un script Bash d’inventaire</b>          | <b>15</b> |
| 5.1      | Le script scripts/inventory.sh . . . . .                           | 15        |
| 5.2      | Exécution réelle . . . . .                                         | 16        |
| 5.3      | Améliorations apportées (travail demandé) . . . . .                | 17        |
| 5.4      | Bonnes pratiques Bash . . . . .                                    | 17        |
| <b>6</b> | <b>Atelier 6 — Automatiser l’arrêt et le démarrage des VM</b>      | <b>18</b> |

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| 6.1       | Le script scripts/vm-power.sh . . . . .                            | 18        |
| 6.2       | Mesures de sécurité ajoutées . . . . .                             | 19        |
| 6.3       | Test de chaque action . . . . .                                    | 19        |
| 6.4       | Journal d'exécution (logs/vm-power.log) . . . . .                  | 20        |
| <b>7</b>  | <b>Atelier 7 — Exploiter un Storage Account</b>                    | <b>20</b> |
| 7.1       | Réutiliser et inspecter le Storage Account . . . . .               | 20        |
| 7.2       | Sécuriser : désactiver l'accès public . . . . .                    | 21        |
| 7.3       | RBAC : control-plane vs data-plane . . . . .                       | 21        |
| 7.4       | Déposer les exports et vérifier la fermeture . . . . .             | 21        |
| 7.5       | Pourquoi le stockage de rapports ne doit pas être public . . . . . | 22        |
| 7.6       | Captures . . . . .                                                 | 22        |
| <b>8</b>  | <b>Atelier 8 — Surveiller les métriques avec Azure Monitor</b>     | <b>23</b> |
| 8.1       | Identifier la VM et lister les métriques . . . . .                 | 24        |
| 8.2       | Lire la métrique CPU . . . . .                                     | 24        |
| 8.3       | Créer une alerte CPU > 80 % . . . . .                              | 24        |
| 8.4       | Justification du seuil . . . . .                                   | 25        |
| 8.5       | Deux autres alertes utiles + limites . . . . .                     | 25        |
| 8.6       | Captures . . . . .                                                 | 26        |
| <b>9</b>  | <b>Atelier 9 — Écrire un script de contrôle de santé</b>           | <b>27</b> |
| 9.1       | Le script scripts/healthcheck.sh . . . . .                         | 27        |
| 9.2       | Preuve d'exécution . . . . .                                       | 28        |
| 9.3       | Vérifications ajoutées . . . . .                                   | 28        |
| <b>10</b> | <b>Atelier 10 — Automatiser avec Python et Azure SDK</b>           | <b>29</b> |
| 10.1      | Installation des dépendances . . . . .                             | 29        |
| 10.2      | Le script python/inventory.py . . . . .                            | 29        |
| 10.3      | Exécution et CSV généré . . . . .                                  | 30        |
| 10.4      | Bash ou Python? . . . . .                                          | 31        |
| 10.5      | Capture . . . . .                                                  | 31        |
| <b>11</b> | <b>Atelier 11 — Analyse FinOps d'exploitation</b>                  | <b>32</b> |
| 11.1      | Ressources les plus coûteuses . . . . .                            | 32        |
| 11.2      | Tableau FinOps . . . . .                                           | 33        |
| 11.3      | Stratégie d'arrêt/démarrage & budget . . . . .                     | 33        |
| 11.4      | Trois recommandations FinOps pour la DSI . . . . .                 | 33        |
| <b>12</b> | <b>Atelier 12 — Rapport d'exploitation ShopEasy</b>                | <b>34</b> |
| 12.1      | Contexte et périmètre . . . . .                                    | 34        |
| 12.2      | Inventaire des ressources . . . . .                                | 34        |
| 12.3      | Tags et gouvernance . . . . .                                      | 34        |
| 12.4      | État des VM et actions réalisées . . . . .                         | 34        |
| 12.5      | Stockage opérationnel . . . . .                                    | 35        |
| 12.6      | Monitoring et alertes . . . . .                                    | 35        |
| 12.7      | Scripts produits . . . . .                                         | 35        |

---

|                                                            |           |
|------------------------------------------------------------|-----------|
| 12.8 Analyse sécurité . . . . .                            | 35        |
| 12.9 Analyse FinOps . . . . .                              | 35        |
| 12.10Recommandations d’amélioration (priorisées) . . . . . | 36        |
| <b>13 Quiz de validation</b>                               | <b>37</b> |

# 1 Atelier 1 — Prise en main d'Azure CLI

🎯 **Objectif** — vérifier l'accès à l'environnement Azure, identifier la subscription active et se positionner sur le bon groupe de ressources.

📦 **Livrable attendu** — extrait de terminal montrant la subscription active, le groupe de ressources cible et les valeurs par défaut configurées.

## 1.1 Contexte — du déploiement à l'exploitation

Le TP1 a **conçu et déployé manuellement** l'architecture ShopEasy ; le TP2 l'a **décrite en Terraform**. Le TP3 ouvre la phase d'**exploitation** : administrer, inventorier, surveiller, automatiser et documenter cet environnement avec **Azure CLI, Bash et Python**. L'environnement de travail est celui du TP2, **redéployé à l'identique** via `terraform apply` (21 ressources) : un groupe de ressources `rg-shopeasy-dev` contenant un réseau segmenté, deux VM Linux derrière un Load Balancer et un compte de stockage privé.

| Périmètre            | Valeur                                                              |
|----------------------|---------------------------------------------------------------------|
| Abonnement           | Azure for Students                                                  |
| Région               | swedencentral (Stockholm)                                           |
| Groupe de ressources | rg-shopeasy-dev                                                     |
| VM web               | vm-shopeasy-dev-web-1, vm-shopeasy-dev-web-2<br>(Standard_B2ats_v2) |
| Storage Account      | shopeasydevdocs350wnq                                               |

### 💡 Contraintes Azure for Students

La région `swedencentral` et la taille `Standard_B2ats_v2` sont conservées du TP1/TP2 : `francecentral` est bloquée par la policy *Allowed regions*, et `Standard_B1s` est indisponible sur l'abonnement étudiant.

## 1.2 Centralisation des variables — `variables.sh`

Toutes les commandes et scripts s'appuient sur un jeu de variables centralisé dans `variables.sh`, à sourcer avant chaque session. Cette factorisation évite de répéter les noms de ressources et permet de rejouer les commandes sur un autre environnement.

```
>_ variables.sh
```

```
#!/usr/bin/env bash
# variables.sh - Variables d'exploitation de l'environnement ShopEasy (TP3)

export RG="rg-shopeasy-dev"           # Groupe de ressources dédié au projet
```

```

export LOCATION="swedencentral"           # Region de deployment
export VM1="vm-shopeasy-dev-web-1"        # Première VM web
export VM2="vm-shopeasy-dev-web-2"        # Seconde VM web
export STORAGE="shopeasydevdocs350wnq"    # Storage Account (suffixe aleatoire Terraform)
export CONTAINER="operations"             # Conteneur des rapports d'exploitation

export ARM_SUBSCRIPTION_ID="$(az account show --query id -o tsv)"
export AZURE_SUBSCRIPTION_ID="$ARM_SUBSCRIPTION_ID" # Utilisée par le SDK Python
export AZURE_RESOURCE_GROUP="$RG"

```

Aucun identifiant n'est codé en dur : la subscription est résolue dynamiquement par `az account show`.

### 1.3 Version d'Azure CLI & subscription active

>\_ bash

```

az version
az account show --output table
az account list --output table

```

>\_ console

azure-cli : 2.87.0

| EnvironmentName | IsDefault | Name               | State   | TenantDefaultDomain | TenantId |
|-----------------|-----------|--------------------|---------|---------------------|----------|
| -----           | -----     | -----              | -----   | -----               | -----    |
| AzureCloud      | True      | Azure for Students | Enabled | efrei.net           | <masque> |

La subscription **Azure for Students** est active (State = Enabled) et définie par défaut (IsDefault = True). Les identifiants (subscription ID, tenant ID) sont **masqués** dans les livrables ; ils ne sont jamais écrits en clair dans un fichier versionné.

### 1.4 Groupe de ressources cible

>\_ bash

```

az group show --name "$RG" \
  --query "{Nom:name, Region:location, Etat:properties.provisioningState}" -o table

```

>\_ console

| Nom             | Region        | Etat      |
|-----------------|---------------|-----------|
| -----           | -----         | -----     |
| rg-shopeasy-dev | swedencentral | Succeeded |

Le groupe `rg-shopeasy-dev` existe, en région `swedencentral`, à l'état `Succeeded` : l'environnement à exploiter est bien en place.

## 1.5 Configuration des valeurs par défaut

Pour éviter de répéter `--resource-group` et `--location` sur chaque commande, on configure des valeurs par défaut au niveau de la CLI.

**>\_ bash**

```
az configure --defaults group="$RG" location="$LOCATION"
az configure --list-defaults --output table
```

**>\_ console**

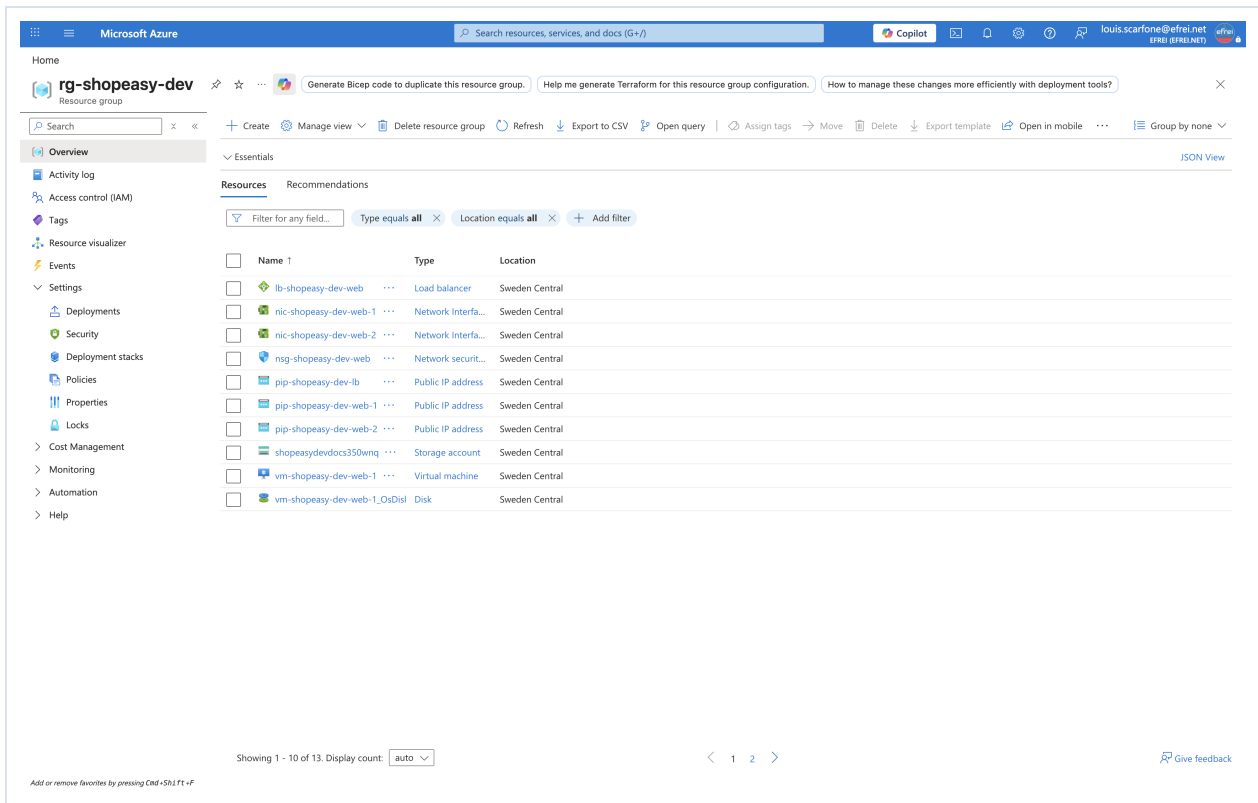
| Name     | Source          | Value           |
|----------|-----------------|-----------------|
| group    | ~/.azure/config | rg-shopeasy-dev |
| location | ~/.azure/config | swedencentral   |

Effet immédiat : les commandes suivantes ciblent le bon groupe **sans -g** ; `az resource list` renvoie alors directement les **13 ressources** de premier niveau.

## 1.6 Travail demandé

1. **Vérifier la subscription active.** `az account show` → **Azure for Students**, *Enabled*, par défaut.
2. **Vérifier que le groupe de ressources existe.** `az group show` → `rg-shopeasy-dev`, `swedencentral`, Succeeded.
3. **Configurer les valeurs par défaut.** `az configure --defaults` → groupe et région enregistrés dans `~/.azure/config`, commandes ultérieures raccourcies.

## 1.7 Capture



Groupe de ressources *rg-shopeasy-dev* en région *Sweden Central* avec ses ressources (Portal → Resource groups → *rg-shopeasy-dev*).

### ✓ État après l'atelier

- Accès Azure validé : subscription **Azure for Students** active (*Enabled*, par défaut).
- Groupe *rg-shopeasy-dev* présent en *swedencentral* (*Succeeded*), contenant les 13 ressources de l'architecture.
- Valeurs par défaut configurées (*group*, *location*) : commandes plus courtes et reproductibles.
- Variables d'exploitation centralisées dans *variables.sh* (aucun secret en dur).
- **Prêt pour l'Atelier 2 — Inventorier les ressources Azure.**

## 2 Atelier 2 — Inventorier les ressources Azure

🎯 **Objectif** — produire un inventaire lisible des ressources ShopEasy, rejouable sans passer par le portail.

📦 **Livrable attendu** — fichiers *resources.json*, *resources.tsv* et tableau d'inventaire complété.



## 2.1 Lister les ressources

L'inventaire est la base de l'exploitation : avant de surveiller, sécuriser ou optimiser, il faut savoir ce qui existe. Azure CLI liste les ressources et choisit le format de sortie selon l'usage.

>\_ bash

```
az resource list --resource-group "$RG" \
  --query "[].{Nom:name,Type:type,Region:location}" --output table
```

>\_ console

| Nom                    | Type                                    | Region        |
|------------------------|-----------------------------------------|---------------|
| vnet-shopeasy-dev      | Microsoft.Network/virtualNetworks       | swedencentral |
| shopeasydevdocs350wnq  | Microsoft.Storage/storageAccounts       | swedencentral |
| pip-shopeasy-dev-web-1 | Microsoft.Network/publicIPAddresses     | swedencentral |
| pip-shopeasy-dev-lb    | Microsoft.Network/publicIPAddresses     | swedencentral |
| nsg-shopeasy-dev-web   | Microsoft.Network/networkSecurityGroups | swedencentral |
| pip-shopeasy-dev-web-2 | Microsoft.Network/publicIPAddresses     | swedencentral |
| lb-shopeasy-dev-web    | Microsoft.Network/loadBalancers         | swedencentral |
| nic-shopeasy-dev-web-2 | Microsoft.Network/networkInterfaces     | swedencentral |
| nic-shopeasy-dev-web-1 | Microsoft.Network/networkInterfaces     | swedencentral |
| vm-shopeasy-dev-web-2  | Microsoft.Compute/virtualMachines       | swedencentral |
| vm-shopeasy-dev-web-1  | Microsoft.Compute/virtualMachines       | swedencentral |
| ..._OsDisk_... (x2)    | Microsoft.Compute/disks                 | swedencentral |

## 2.2 Exporter l'inventaire (JSON + TSV)

Un inventaire d'exploitation doit être **conservé et rejouable**. On exporte le **JSON** complet (archivage, traitement) et le **TSV** (réutilisation tableur/script).

>\_ bash

```
az resource list --resource-group "$RG" --output json > exports/resources.json
az resource list --resource-group "$RG" \
  --query "[].{Nom:name,Type:type,Region:location}" --output tsv > exports/resources.tsv
```

Les ressources portent déjà des **tags hérités du TP2** (appliqués par Terraform, en snake\_case). L'Atelier 3 normalisera une stratégie de tags d'exploitation.

## 2.3 Identifier les types

>\_ bash

```
az resource list --resource-group "$RG" --query "[].type" -o tsv | sort | uniq -c | sort -rn
```

>\_ console

```
3 Microsoft.Network/publicIPAddresses
2 Microsoft.Network/networkInterfaces
```

```

2 Microsoft.Compute/virtualMachines
2 Microsoft.Compute/disks
1 Microsoft.Storage/storageAccounts
1 Microsoft.Network/virtualNetworks
1 Microsoft.Network/networkSecurityGroups
1 Microsoft.Network/loadBalancers

```

8 types pour 13 ressources de premier niveau.

### ⚠ Ressources de premier niveau seulement

az resource list ne renvoie que les ressources de premier niveau. Les sous-ressources (subnet, règles NSG, règle/sonde/backend pool du Load Balancer) sont imbriquées dans leur parent. Côté Terraform, l'état gère **21 objets**.

## 2.4 Tableau d'inventaire

| Ressource             | Type          | Rôle dans l'architecture                                                |
|-----------------------|---------------|-------------------------------------------------------------------------|
| vnet-shopeasy-dev     | VNet          | Réseau privé 10.20.0.0/16, segmentation (subnet snet-web).              |
| nsg-shopeasy-dev-web  | NSG           | Pare-feu du subnet : HTTP (80) ouvert, SSH (22) restreint à l'IP admin. |
| lb-shopeasy-dev-web   | Load Balancer | LB <b>Standard</b> L4 : répartit le trafic HTTP sur les 2 VM.           |
| pip-shopeasy-dev-lb   | Public IP     | IP publique du LB = point d'entrée HTTP unique.                         |
| pip-...-web-1/2       | Public IP     | IP publiques d'administration des 2 VM.                                 |
| nic-...-web-1/2       | NIC           | Cartes réseau des 2 VM (subnet + backend pool).                         |
| vm-...-web-1/2        | VM            | Serveurs web : Ubuntu 22.04 + Nginx (Standard_B2ats_v2).                |
| ..._osDisk_... (×2)   | Disk          | Disques OS managés des 2 VM.                                            |
| shopeasydevdocs350wnq | Storage       | Stockage documentaire privé (StorageV2, LRS, versioning).               |

## 2.5 Travail demandé

1. **Tableau d'inventaire (nom, type, région, rôle)**. Réalisé : 13 ressources, toutes en swedencentral, reliées à leur rôle.
2. **Inventaire rejouable sans le portail**. Produit en CLI, exporté en `resources.json` et `resources.tsv`, industrialisé ensuite dans `inventory.sh` (Atelier 5).

### ✓ État après l'atelier

- Inventaire CLI complet : **13 ressources, 8 types**, région unique `swedencentral`.
- Exports générés : `resources.json` (368 lignes) et `resources.tsv` (13 lignes).
- Tableau d'inventaire rôle par rôle établi (base du rapport d'exploitation).
- Constat : ressources déjà taguées (TP2), à normaliser à l'Atelier 3.
- **Prêt pour l'Atelier 3 — Normaliser les tags d'exploitation.**

## 3 Atelier 3 — Normaliser les tags d'exploitation

🎯 **Objectif** — appliquer une stratégie de tags minimale pour faciliter l'exploitation, le suivi des coûts et l'identification des responsabilités.

📦 **Livrable attendu** — export JSON (et capture) montrant les tags du groupe de ressources et d'au moins une VM.

### 3.1 Stratégie de tags d'exploitation

Les ressources avaient déjà des **tags techniques hérités du TP2** (posés par Terraform, en `snake_case`). On les **normalise** en une convention d'exploitation homogène (`PascalCase`).

| Tag         | Valeur            | Rôle                                         |
|-------------|-------------------|----------------------------------------------|
| Application | shopeasy          | Relier la ressource à l'application métier.  |
| Environment | dev               | Distinguer dev / test / prod.                |
| Owner       | cloudops-shopeasy | Identifier l'équipe responsable.             |
| CostCenter  | cloud-training    | Imputer les coûts (FinOps).                  |
| ManagedBy   | mixed             | Provisionné par Terraform, exploité par CLI. |
| Role (VM)   | web               | Rôle technique de la VM.                     |

### 💡 Choix ManagedBy = mixed

L'infrastructure a été créée en IaC (Terraform, TP2) mais est administrée en CLI au TP3. La valeur `mixed` décrit honnêtement ce cycle de vie hybride, plutôt que `cli` (qui masquerait l'origine IaC) ou `terraform` (qui ignorerait l'exploitation CLI).

### 3.2 État avant normalisation

Le RG et les VM portaient les 5 tags Terraform en `snake_case` :

**>\_ console**

```
{
  "cost_center": "cloud-training",
  "environment": "dev",
  "managed_by": "terraform",
  "owner": "formation",
  "project": "shopeasy"
}
```

### 3.3 Appliquer les tags (RG puis VM)

az group update --tags **remplace** l'intégralité du jeu de tags du RG. Pour les VM, az resource tag --ids remplace proprement et ajoute Role=web.

**>\_ bash**

```
az group update --name "$RG" \
  --tags Application=shopeasy Environment=dev Owner=cloudops-shopeasy \
    CostCenter=cloud-training ManagedBy=mixed

for VM in "$VM1" "$VM2"; do
  VM_ID=$(az vm show -g "$RG" -n "$VM" --query id -o tsv)
  az resource tag --ids "$VM_ID" \
    --tags Application=shopeasy Environment=dev Owner=cloudops-shopeasy \
      CostCenter=cloud-training ManagedBy=mixed Role=web
done
```

**>\_ console**

| Application | Environment | Owner             | CostCenter     | ManagedBy |
|-------------|-------------|-------------------|----------------|-----------|
| shopeasy    | dev         | cloudops-shopeasy | cloud-training | mixed     |

### 3.4 Vérification et couverture

Le travail demandé porte sur le RG et les VM. Les autres ressources conservent les tags techniques Terraform :

**>\_ bash**

```
az resource list -g "$RG" --query "[?tags.Application==null].name" -o tsv
```

**11 ressources** ne portent pas encore le tag Application normalisé. Ce manque est **volontairement laissé visible** : il est **détecté automatiquement** par inventory.sh (Atelier 5) et healthcheck.sh (Atelier 9), et son extension figure dans les recommandations FinOps (Atelier 11).

#### ⚠ Drift Terraform

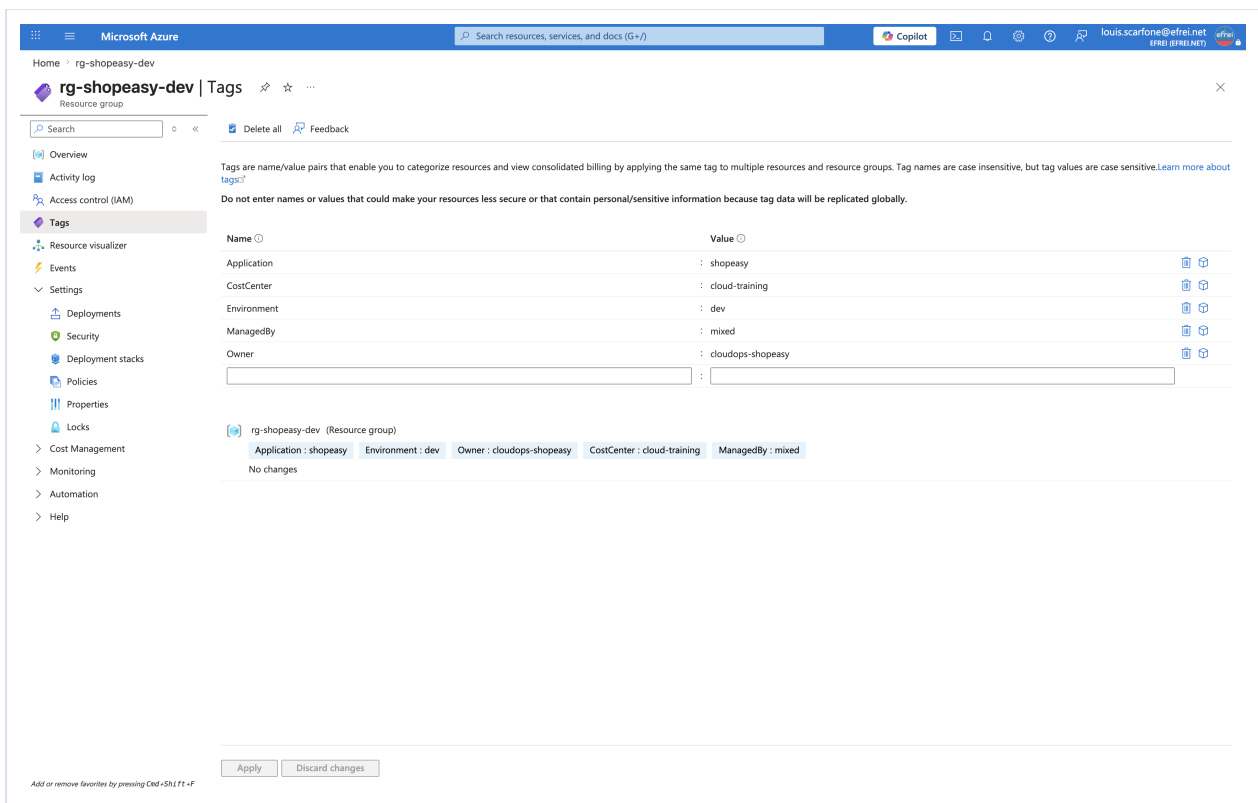
Modifier les tags via CLI sur des ressources gérées par Terraform crée une **dérive (drift)** : au prochain terraform plan, Terraform proposerait de restaurer ses tags snake\_case (phé-

nomène étudié au TP2, Atelier 11). Réconciliation : reporter la convention dans le code (`locals.common_tags`) ou exclure les tags (`lifecycle ignore_changes`). Pour le TP3, aucun apply n'est rejoué d'ici la destruction finale.

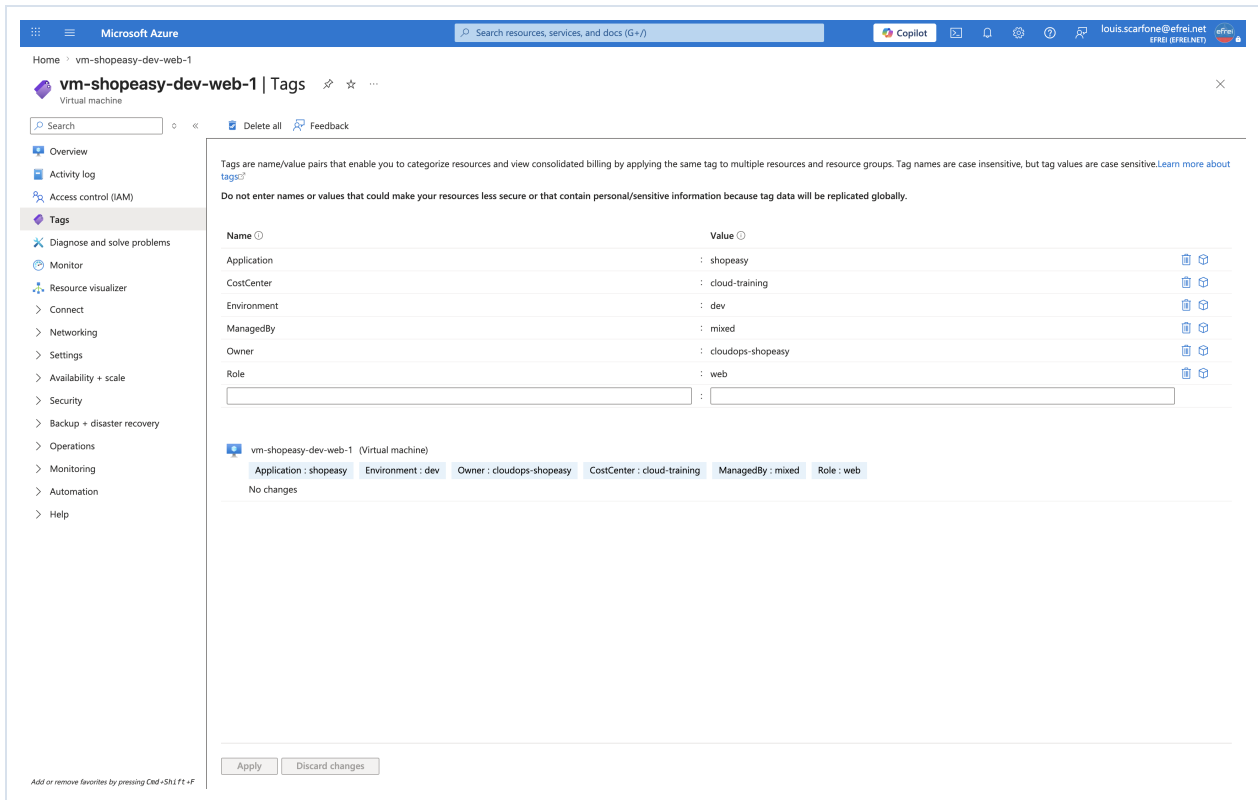
### 3.5 Pourquoi les tags sont utiles (FinOps & gouvernance)

**FinOps** — les tags `CostCenter`, `Application` et `Environment` permettent de **ventiler la facture** Azure (Cost Management) par centre de coût, application et environnement. Sans eux, impossible de répondre à « combien coûte ShopEasy en dev ? ». **Gouvernance** — `Owner` identifie le responsable (incident, autorisation d'arrêt), `ManagedBy` documente le mode de gestion ; les tags servent de base à des **Azure Policy** et au filtrage des inventaires.

### 3.6 Captures



Tags normalisés du groupe `rg-shopeasy-dev` (Portal → Resource groups → `rg-shopeasy-dev` → Tags) : les 5 tags d'exploitation.



Tags normalisés de la VM `vm-shopeasy-dev-web-1` (Portal → Virtual machines → ... → Tags) : les 6 tags dont `Role=web`.

### ✓ État après l'atelier

- Convention normalisée (Application, Environment, Owner, CostCenter, ManagedBy) appliquée au RG et aux 2 VM (+ Role=web).
- Anciennes clés `snake_case` remplacées proprement; preuve exportée (`tags-rg.json`, `tags-vm.json`).
- **11 ressources** encore en `snake_case` : manque tracé, à corriger via inventaire automatisé.
- Drift Terraform identifié et documenté (réconciliation par le code ou `ignore_changes`).
- **Prêt pour l'Atelier 4 — Administrer les machines virtuelles.**

## 4 Atelier 4 — Administrer les machines virtuelles

🎯 **Objectif** — réaliser les opérations courantes sur les VM : lister, vérifier l'état, démarrer, arrêter, désallouer, redémarrer et exécuter une commande distante.

📦 **Livrable attendu** — tableau des VM (état, IP, taille, action), avec la commande utilisée et le résultat observé.

### 4.1 Lister les VM et leur état

--show-details ajoute l'état d'alimentation et l'IP publique.

**>\_ bash**

```
az vm list -g "$RG" --show-details \
  --query "[].{Nom:name,Etat:powerState,IP:publicIps,Taille:hardwareProfile.vmSize}" -o table
```

**>\_ console**

| Nom                   | Etat       | IP             | Taille            |
|-----------------------|------------|----------------|-------------------|
| vm-shopeasy-dev-web-1 | VM running | 20.240.233.166 | Standard_B2ats_v2 |
| vm-shopeasy-dev-web-2 | VM running | 20.91.246.70   | Standard_B2ats_v2 |

## 4.2 Exécuter une commande distante (vérifier le serveur web)

az vm run-command invoke exécute un script dans la VM via l'agent Azure, **sans connexion SSH** (utile quand le port 22 est restreint à l'IP admin).

**>\_ bash**

```
az vm run-command invoke -g "$RG" -n "$VM1" --command-id RunShellScript \
  --scripts "hostname && uptime && systemctl is-active nginx && \
    curl -s -o /dev/null -w 'HTTP local: %{http_code}\n' http://localhost" \
  --query "value[0].message" -o tsv
```

**>\_ console**

```
[stdout]
vm-shopeasy-dev-web-1
08:15:51 up 21 min, 0 users, load average: 0.00, 0.00, 0.00
active
nginx.service - active (running) since Fri 2026-06-26 07:55:02 UTC
HTTP local: 200
```

**Lecture :** le serveur répond, nginx est active (running) et la requête HTTP locale renvoie **200**. Le service web est opérationnel.

## 4.3 Redémarrer et désallouer

Deux actions d'exploitation sur deux VM distinctes : **redémarrer** la VM 1 (reste allouée) et **désallouer** la VM 2 non utilisée (geste FinOps : libère le calcul).

**>\_ bash**

```
az vm restart -g "$RG" -n "$VM1" # redemarrage, VM toujours allouee
az vm deallocate -g "$RG" -n "$VM2" # arret + liberation du compute
```

**>\_ console**

| Nom                   | Etat       | IP             | Taille            |
|-----------------------|------------|----------------|-------------------|
| vm-shopeasy-dev-web-1 | VM running | 20.240.233.166 | Standard_B2ats_v2 |

```
vm-shopeasy-dev-web-2 VM deallocated 20.91.246.70 Standard_B2ats_v2
```

#### 4.4 stop ≠ deallocate (point FinOps essentiel)

| Commande         | Effet                                                              | Compute facturé |
|------------------|--------------------------------------------------------------------|-----------------|
| az vm stop       | Arrête l'OS, mais la VM <b>reste allouée</b>                       | <b>Continue</b> |
| az vm deallocate | Arrête <b>et libère</b> le calcul ( <i>Stopped (deallocated)</i> ) | <b>Stoppé</b>   |

##### ⚠ Ce qui reste facturé même désallouée

Le **disque OS** et l'**IP publique Standard statique** (réservée) restent facturés : vm-shopeasy-dev-web-2 affiche encore son IP bien qu'éteinte. La suppression des IP publiques (Bastion) est un levier FinOps de l'Atelier 11.

#### 4.5 Tableau récapitulatif des VM

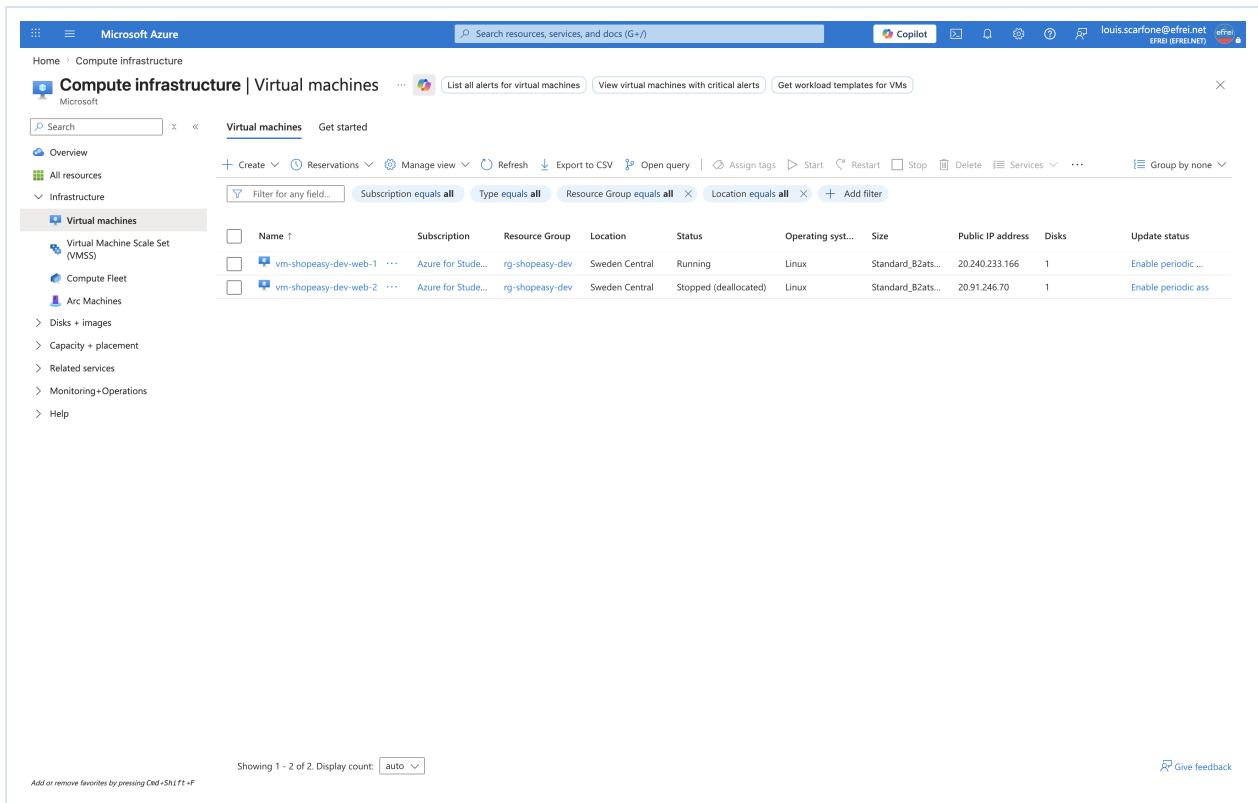
| VM           | Taille   | État final         | Action & résultat                              |
|--------------|----------|--------------------|------------------------------------------------|
| vm-...-web-1 | B2ats_v2 | <b>running</b>     | run-command (Nginx active, HTTP 200) + restart |
| vm-...-web-2 | B2ats_v2 | <b>deallocated</b> | deallocate (compute libéré, FinOps)            |

#### 4.6 Travail demandé

1. **Lister toutes les VM.** az vm list → 2 VM.
2. **Identifier taille, IP, état.** --show-details → B2ats\_v2, IP publiques, VM running.
3. **Redémarrer une VM.** az vm restart → reste running.
4. **Désallouer une VM non utilisée.** az vm deallocate → VM deallocated.
5. **Commande distante.** az vm run-command invoke → Nginx active, HTTP 200.



## 4.7 Capture



Liste des VM (Portal → Virtual machines) : web-1 Running et web-2 Stopped (deallocated) après les actions.

### ✓ État après l'atelier

- 2 VM Standard\_B2ats\_v2 inventoriées (état, IP, taille).
- Commande distante validée (sans SSH) : web-1 sert Nginx, HTTP 200.
- restart sur la VM 1 (running), deallocate sur la VM 2 (deallocated, compute libéré).
- Distinction stop / deallocate documentée (impact FinOps).
- **Prêt pour l'Atelier 5 — Construire un script Bash d'inventaire.**

## 5 Atelier 5 — Construire un script Bash d'inventaire

- 🎯 **Objectif** — transformer les commandes manuelles d'inventaire en script réutilisable.
- 📦 **Livrable attendu** — script `inventory.sh` fonctionnel, captures d'exécution et fichiers générés dans `exports/`.

### 5.1 Le script `scripts/inventory.sh`

Le script reprend les commandes des Ateliers 2 à 4 et y ajoute une **synthèse chiffrée** et des **contrôles d'exploitation**. Paramétrable, robuste (`set -euo pipefail`), exports **horodatés**.

**>\_ scripts/inventory.sh**

```
#!/usr/bin/env bash
set -euo pipefail

RG="${1:-rg-shopeasy-dev}"          # groupe de ressources (argument ou default)
DATE="$(date +%Y%m%d-%H%M%S)"      # horodatage des fichiers d'export
OUT_DIR="exports"; mkdir -p "$OUT_DIR"

# 1. Verification de l'existence du groupe de ressources
if ! az group show --name "$RG" >/dev/null 2>&1; then
    echo "ERREUR : le groupe de ressources '$RG' est introuvable." >&2; exit 1
fi

# 2. Export JSON des ressources
az resource list --resource-group "$RG" \
    --query "[].{name:name,type:type,location:location,tags:tags,id:id}" \
    --output json > "$OUT_DIR/resources-$DATE.json"

# 3. Export TXT de l'etat des VM
az vm list --resource-group "$RG" --show-details \
    --query "[].{name:name,powerState:powerState,publicIps:publicIps,vmSize:hardwareProfile.vmSize}" \
    --output table > "$OUT_DIR/vms-$DATE.txt"

# 4. Synthese chiffrée
TOTAL=$(az resource list -g "$RG" --query "length(@)" -o tsv)
NB_VM=$(az vm list -g "$RG" --query "length(@)" -o tsv)
NB_STORAGE=$(az storage account list -g "$RG" --query "length(@)" -o tsv)
NB_UNTAGGED=$(az resource list -g "$RG" --query "length([?tags.Application==null])" -o tsv)
printf " %-34s %s\n" "Ressources (total)" "$TOTAL"
printf " %-34s %s\n" "Machines virtuelles" "$NB_VM"
printf " %-34s %s\n" "Comptes de stockage" "$NB_STORAGE"
printf " %-34s %s\n" "Ressources sans tag Application" "$NB_UNTAGGED"

# 5. Controles : VM en cours d'execution (compute facture)
RUNNING=$(az vm list -g "$RG" --show-details \
    --query "[?powerState=='VM running'].name" -o tsv)
if [[ -n "$RUNNING" ]]; then
    echo " [AVERTISSEMENT] VM en cours d'execution (compute facture) :"
    echo "$RUNNING" | sed 's/^/ - /'
fi
if [[ "$NB_UNTAGGED" -gt 0 ]]; then
    echo " [INFO] $NB_UNTAGGED ressource(s) sans tag 'Application'"
fi
```

## 5.2 Exécution réelle

**>\_ console**

```
[1/4] Export des ressources -> exports/resources-20260626-103147.json
[2/4] Export de l'etat des VM -> exports/vms-20260626-103147.txt
[3/4] Calcul de la synthese...

--- Synthèse -----
Ressources (total)           13
Machines virtuelles          2
Comptes de stockage          1
Ressources sans tag Application 11
```

```
-----
[4/4] Controles d'exploitation...
```

```
[AVERTISSEMENT] VM en cours d'execution (compute facture) :
```

```
- vm-shopeasy-dev-web-1
```

```
[INFO] 11 ressource(s) sans tag 'Application'
```

### 💡 Validation croisée

L'avertissement ne liste **que** vm-shopeasy-dev-web-1 (en VM running), et **pas** vm-shopeasy-dev-web-2 (désallouée à l'Atelier 4). Le filtre powerState=='VM running' fonctionne donc correctement.

## 5.3 Améliorations apportées (travail demandé)

| Amélioration demandée           | Implémentation (JMESPath)             | Résultat       |
|---------------------------------|---------------------------------------|----------------|
| Nombre total de ressources      | length(@)                             | 13             |
| Nombre de VM                    | az vm list ... length(@)              | 2              |
| Nombre de comptes de stockage   | az storage account list ... length(@) | 1              |
| Ressources sans tag Application | length([?tags.Application==null])     | 11             |
| Avertissement si VM running     | [?powerState=='VM running']           | web-1 signalée |

## 5.4 Bonnes pratiques Bash

| Bonne pratique         | Mise en œuvre                                                               |
|------------------------|-----------------------------------------------------------------------------|
| Arrêt sur erreur       | set -euo pipefail (erreur, variable non définie, échec dans un pipe).       |
| Paramétrage            | RG="\${1:-rg-shopeasy-dev}" : groupe en argument, valeur par défaut.        |
| Vérification préalable | az group show ...    exit 1 : sortie propre si le RG n'existe pas.          |
| Traçabilité            | Exports <b>horodatés</b> (-\$DATE) : pas d'écrasement, historique conservé. |
| Idempotence            | mkdir -p, relances sans effet de bord.                                      |

### ✅ État après l'atelier

- Script scripts/inventory.sh fonctionnel, commenté, robuste et paramétrable.
- Synthèse automatisée : **13 ressources, 2 VM, 1 compte de stockage, 11 sans tag**

**Application.**

- Avertissement FinOps opérationnel (VM running détectée), filtrage d'état validé.
- Exports horodatés générés dans exports/ (JSON ressources + TXT VM).
- Prêt pour l'Atelier 6 — Automatiser l'arrêt et le démarrage des VM.

## 6 Atelier 6 — Automatiser l'arrêt et le démarrage des VM

🎯 **Objectif** — construire un script pilotant l'état des VM (start / stop / deallocate / status) de manière sûre.

📦 **Livrable attendu** — script `vm-power.sh`, journal d'exécution et explication des mesures de sécurité ajoutées.

### 6.1 Le script `scripts/vm-power.sh`

>\_ `scripts/vm-power.sh`

```
#!/usr/bin/env bash
set -euo pipefail
RG="{1:-}"; ACTION="{2:-}"; ASSUME_YES="{3:-}" # -y saute la confirmation

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
LOG_FILE="$(dirname "$SCRIPT_DIR")/logs/vm-power.log"; mkdir -p "$(dirname "$LOG_FILE")"
log() { echo "$(date '+%Y-%m-%d %H:%M:%S') | $1" | tee -a "$LOG_FILE"; }

[[ -z "$RG" || -z "$ACTION" ]] && { echo "Usage: $0 <rg> <start|stop|deallocate|status> [-y]"; exit 1;
↩ }

# Garde-fou 1 : interdiction d'agir sur un RG de production
if [[ "$RG" == *prod* ]]; then
    log "REFUS : action '$ACTION' interdite sur un RG de production ('$RG')."; exit 1
fi
az group show --name "$RG" >/dev/null 2>&1 || { log "ERREUR : RG introuvable."; exit 1; }

# Garde-fou 2 : confirmation avant action destructive
confirm() {
    [[ "$ASSUME_YES" == "-y" ]] && return 0
    read -r -p "Confirmer '$ACTION' sur TOUTES les VM de '$RG' ? (oui/non) " ANS || true
    [[ "$ANS" == "oui" ]]
}

VMS=$(az vm list -g "$RG" --query "[].name" -o tsv)
case "$ACTION" in stop|deallocate)
    confirm || { log "Action '$ACTION' ANNULEE par l'utilisateur."; exit 0; } ;;
esac

log "=== Action '$ACTION' sur RG '$RG' ==="
for VM in $VMS; do
    case "$ACTION" in
        start)      az vm start      -g "$RG" -n "$VM" -o none; log " start      -> $VM : OK" ;;
        stop)       az vm stop       -g "$RG" -n "$VM" -o none; log " stop       -> $VM : OK" ;;
    esac
done
```

```

deallocate) az vm deallocate -g "$RG" -n "$VM" -o none; log " deallocate -> $VM : OK" ;;
status)    STATE=$(az vm get-instance-view -g "$RG" -n "$VM" \
    --query "instanceView.statuses[?starts_with(code,'PowerState/')].displayStatus" -o
    ↪ tsv)
    log " status -> $VM : $STATE" ;;
esac
done
log "=== Termine ==="

```

## 6.2 Mesures de sécurité ajoutées

| Mesure                   | Mise en œuvre                          | Pourquoi                                                        |
|--------------------------|----------------------------------------|-----------------------------------------------------------------|
| Garde-fou production     | if [[ "\$RG" == *prod* ]]              | Empêche tout arrêt accidentel d'un environnement de production. |
| Confirmation destructive | fonction confirm() sur stop/deallocate | Évite une coupure de service par erreur de frappe.              |
| Bypass contrôlé          | option -y                              | Permet l'automatisation (cron, CI) en connaissance de cause.    |
| Journalisation           | log() → logs/vm-power.log              | Traçabilité : qui a fait quoi, quand.                           |
| Robustesse               | set -euo pipefail, vérif du RG         | Arrêt propre en cas d'erreur.                                   |

## 6.3 Test de chaque action

>\_ console

```

$ ./vm-power.sh rg-shopeasy-prod deallocate
... | REFUS : action 'deallocate' interdite sur un RG de production. (code 1)

$ echo "non" | ./vm-power.sh rg-shopeasy-dev deallocate
... | Action 'deallocate' ANNULEE par l'utilisateur.

$ echo "oui" | ./vm-power.sh rg-shopeasy-dev stop
WARNING: About to power off the specified VM...
It will continue to be billed. To deallocate a VM, run: az vm deallocate.
... | stop -> vm-shopeasy-dev-web-1 : OK

```

### 💡 Confirmation FinOps par Azure lui-même

Sur stop, Azure CLI affiche « *It will continue to be billed. To deallocate a VM, run : az vm deallocate.* » — confirmant la distinction stop (facturé) / deallocate (compute libéré).

## 6.4 Journal d'exécution (logs/vm-power.log)

>\_ logs/vm-power.log

```
2026-06-26 10:37:00 | status      -> vm-shopeasy-dev-web-1 : VM running
2026-06-26 10:37:03 | REFUS : action 'deallocate' interdite sur un RG de production.
2026-06-26 10:37:05 | Action 'deallocate' ANNULEE par l'utilisateur.
2026-06-26 10:38:23 | stop        -> vm-shopeasy-dev-web-1 : OK (VM arretee, toujours allouee)
2026-06-26 10:39:46 | deallocate  -> vm-shopeasy-dev-web-1 : OK (compute libere)
2026-06-26 10:41:18 | status      -> vm-shopeasy-dev-web-1 : VM running
```

À l'issue des tests, l'environnement est remis à l'état nominal (les 2 **VM running**).

### ✓ État après l'atelier

- Script scripts/vm-power.sh : 4 actions (start/stop/deallocate/status), toutes testées.
- 3 mesures de sécurité : garde-fou prod, confirmation destructive (+ bypass -y), journalisation horodatée.
- Distinction stop / deallocate confirmée par l'avertissement natif d'Azure CLI.
- Journal logs/vm-power.log produit (26 lignes) ; environnement remis à l'état nominal.
- **Prêt pour l'Atelier 7 — Exploiter un Storage Account.**

## 7 Atelier 7 — Exploiter un Storage Account

🎯 **Objectif** — disposer d'un espace de stockage opérationnel et sécurisé pour déposer les rapports et exports d'exploitation.

📦 **Livrable attendu** — sortie CLI (et capture) montrant le conteneur, les fichiers déposés et l'accès public désactivé.

### 7.1 Réutiliser et inspecter le Storage Account

Le compte shopeasydevdocs350wnq (créé au TP2) est **réutilisé**. Inspection initiale :

>\_ console

| Name                  | Sku          | AllowBlobPublicAccess | MinimumTlsVersion | HttpsOnly |
|-----------------------|--------------|-----------------------|-------------------|-----------|
| shopeasydevdocs350wnq | Standard_LRS | True                  | TLS1_2            | True      |

**Constat** : TLS 1.2 et HTTPS-only sont corrects, mais **AllowBlobPublicAccess=True** : le provider Terraform laisse ce paramètre activé par défaut — à corriger en exploitation.

## 7.2 Sécuriser : désactiver l'accès public

>\_ bash

```
az storage account update --name "$STORAGE" -g "$RG" --allow-blob-public-access false
az storage container create --account-name "$STORAGE" --name "$CONTAINER" \
  --auth-mode login --public-access off
```

À partir d'ici, **aucun conteneur ne peut être rendu public**, même par erreur. Le conteneur privé operations est créé.

## 7.3 RBAC : control-plane vs data-plane

Le dépôt de blobs avec `--auth-mode login` a d'abord été **refusé** :

>\_ console

```
ERROR: You do not have the required permissions needed to perform this operation.
"Storage Blob Data Contributor" ...
```

Être *Owner* de l'abonnement donne les droits sur le **control-plane** (créer/configurer via ARM), mais **pas** sur le **data-plane** (lire/écrire le contenu des blobs). La bonne pratique — conforme au moindre privilège — est d'**assigner un rôle RBAC data-plane** précis, au scope du compte (pas de clé partagée) :

>\_ bash

```
az role assignment create --role "Storage Blob Data Contributor" \
  --assignee-object-id "$USER_OID" --assignee-principal-type User \
  --scope "$STORAGE_ID"
```

## 7.4 Déposer les exports et vérifier la fermeture

>\_ console

```
$ az storage blob list --account-name "$STORAGE" -c operations --auth-mode login -o table
Nom                                     TailleOctets
-----
inventaires/resources.json             12014
inventaires/resources.tsv               983
inventaires/vms-20260626-103147.txt     281
rapports/rapport-exploitation-shopeasy.md 9772

$ curl -s -o /dev/null -w "%{http_code}" \
  "https://$STORAGE.blob.core.windows.net/operations/inventaires/resources.tsv"
409 -> <Code>PublicAccessNotPermitted</Code>
```

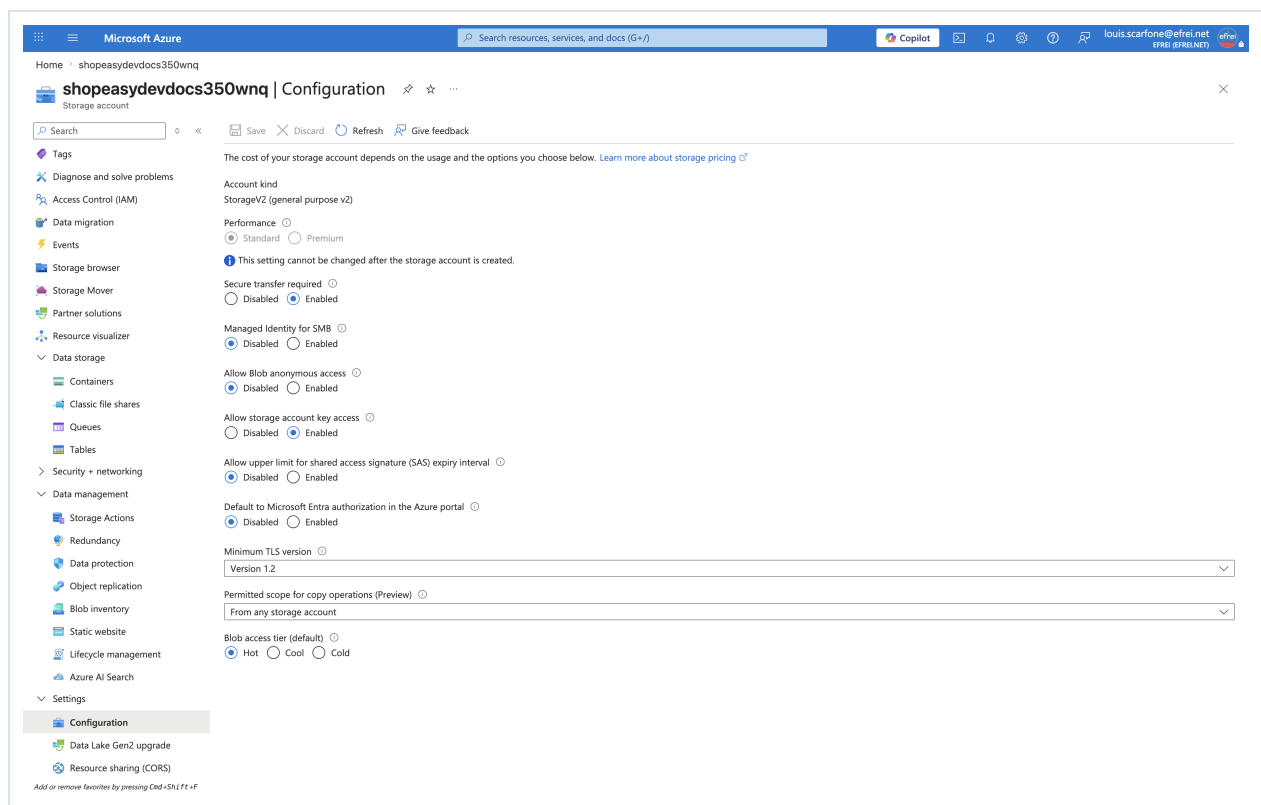
### ⚠ Preuve d'accès fermé

Même en connaissant l'URL exacte, un visiteur anonyme reçoit **409 PublicAccessNotPermitted** : le stockage est bien fermé. L'accès n'est possible qu'avec une identité Azure autorisée (RBAC).

## 7.5 Pourquoi le stockage de rapports ne doit pas être public

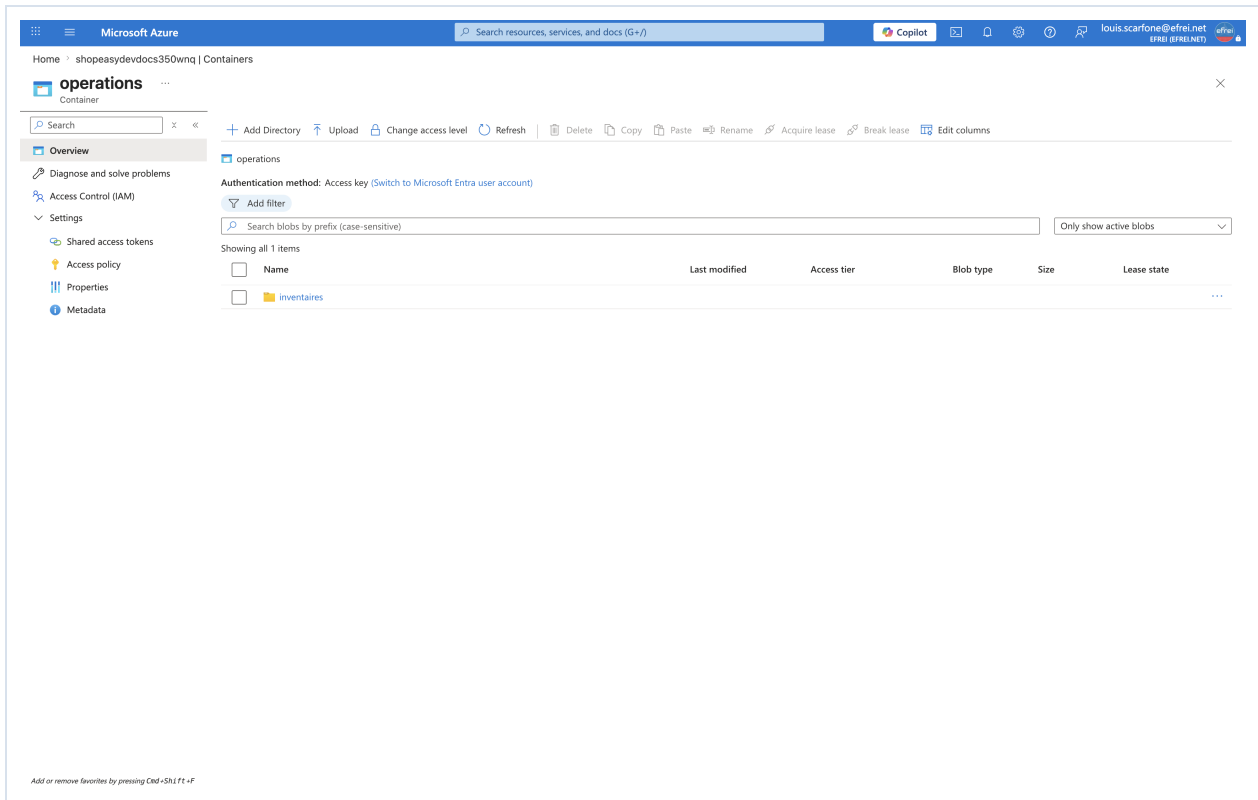
- **Cartographie offerte à un attaquant** : un rapport liste noms de ressources, IP, état des VM, règles réseau.
- **Fuite de données / RGPD** : un blob public est accessible à quiconque connaît l'URL, sans traçabilité.
- **Moindre privilège** : seuls les exploitants **authentifiés** (RBAC) doivent lire ces rapports.
- **Conformité** : « pas de stockage public » est un contrôle standard (CIS Azure).

## 7.6 Captures



*Configuration du compte (Portal → Storage accounts → ... → Configuration) : Allow blob anonymous access = Disabled.*





Conteneur operations et le dossier inventaires/ (exports déposés). Le niveau « Private » n'apparaît plus car l'accès anonyme est désactivé au niveau du compte.

### ✓ État après l'atelier

- Storage Account réutilisé et **sécurisé** : accès public **désactivé** (était True), TLS 1.2, HTTPS-only.
- Conteneur privé operations créé; 4 exports déposés sous inventaires/.
- RBAC data-plane appliqué proprement (Storage Blob Data Contributor, scope = compte) — pas de clé partagée.
- Accès anonyme **prouvé refusé** (HTTP 409 PublicAccessNotPermitted).
- **Prêt pour l'Atelier 8 — Surveiller les métriques avec Azure Monitor.**

## 8 Atelier 8 — Surveiller les métriques avec Azure Monitor

🎯 **Objectif** — interroger les métriques Azure et créer une alerte simple sur une ressource critique.

📦 **Livrable attendu** — sortie CLI montrant la métrique CPU et la règle d'alerte créée, avec justification du seuil.

## 8.1 Identifier la VM et lister les métriques

>\_ bash

```
VM_ID=$(az vm show -g "$RG" -n "$VM1" --query id -o tsv)
az monitor metrics list-definitions --resource "$VM_ID" \
  --query "[].{Metric:name.value, Unite:unit}" -o table
```

>\_ console

| Metric                   | Unite                    |
|--------------------------|--------------------------|
| Percentage CPU           | Percent                  |
| Network In / Network Out | Bytes                    |
| Disk Read Bytes / Write  | Bytes                    |
| CPU Credits Remaining    | Count                    |
| CPU Credits Consumed     | Count                    |
| ...                      | (~30 metriques au total) |

### 💡 Spécificité Standard\_B2ats\_v2 (burstable)

C'est une VM *burstable* (famille B). Les métriques CPU Credits Remaining et CPU Credits Consumed sont propres à cette famille : si le CPU reste élevé, la VM épuise ses crédits et se fait **brider** (throttling).

## 8.2 Lire la métrique CPU

>\_ console

| Heure                | CPU_moyen_pct |
|----------------------|---------------|
| 2026-06-26T08:42:00Z | 0.15          |
| 2026-06-26T08:46:00Z | 0.62          |
| 2026-06-26T08:55:00Z | 0.12          |
| 2026-06-26T09:00:00Z | 0.15          |

Le CPU moyen oscille entre **0,12 %** et **0,62 %** : la VM web est **au repos** (pas de trafic). La métrique fournit une **base de référence** pour calibrer le seuil d'alerte.

## 8.3 Créer une alerte CPU > 80 %

>\_ bash

```
az monitor metrics alert create --name "alert-cpu-high-$VM1" -g "$RG" \
  --scopes "$VM_ID" --condition "avg Percentage CPU > 80" \
  --description "Alerte CPU elevee sur VM web ShopEasy" \
  --evaluation-frequency 1m --window-size 5m --severity 3
```

| > console                            |        |          |             |
|--------------------------------------|--------|----------|-------------|
| Nom                                  | Active | Severite | Condition   |
| -----                                | -----  | -----    | -----       |
| alert-cpu-high-vm-shopeasy-dev-web-1 | True   | 3        | GreaterThan |

## 8.4 Justification du seuil

| Paramètre  | Valeur  | Justification                                                                                       |
|------------|---------|-----------------------------------------------------------------------------------------------------|
| Seuil      | > 80 %  | Assez haut pour ignorer les pics normaux, assez bas pour détecter une saturation avant dégradation. |
| Agrégation | Average | On surveille une charge <b>soutenue</b> , pas un pic instantané.                                    |
| Fenêtre    | 5 min   | <b>Lisse</b> les pics transitoires (charge élevée prolongée requise).                               |
| Fréquence  | 1 min   | Réactivité : réévaluation chaque minute.                                                            |
| Sévérité   | 3 (Low) | CPU élevé = signal d'attention, pas une panne (Sev 0/1 = indispo).                                  |

## 8.5 Deux autres alertes utiles + limites

| Alerte proposée                                 | Pourquoi                                                                          | Sévérité |
|-------------------------------------------------|-----------------------------------------------------------------------------------|----------|
| Disponibilité backends (LB Health Probe Status) | Une VM ne répond plus à la sonde HTTP : indicateur proche de l'expérience client. | 1        |
| Saturation disque / mémoire (VM)                | Disque plein (logs Nginx) ou mémoire saturée bloque le service.                   | 2        |

### ⚠ Limites d'une alerte mal calibrée

**Trop sensible** (seuil bas, fenêtre courte) → faux positifs, *fatigue d'alerte*, vrais incidents ratés.  
**Trop large** (seuil haut, fenêtre longue) → détection tardive. Une alerte doit être **actionnable** et associée à une **procédure** (runbook).

## 8.6 Captures

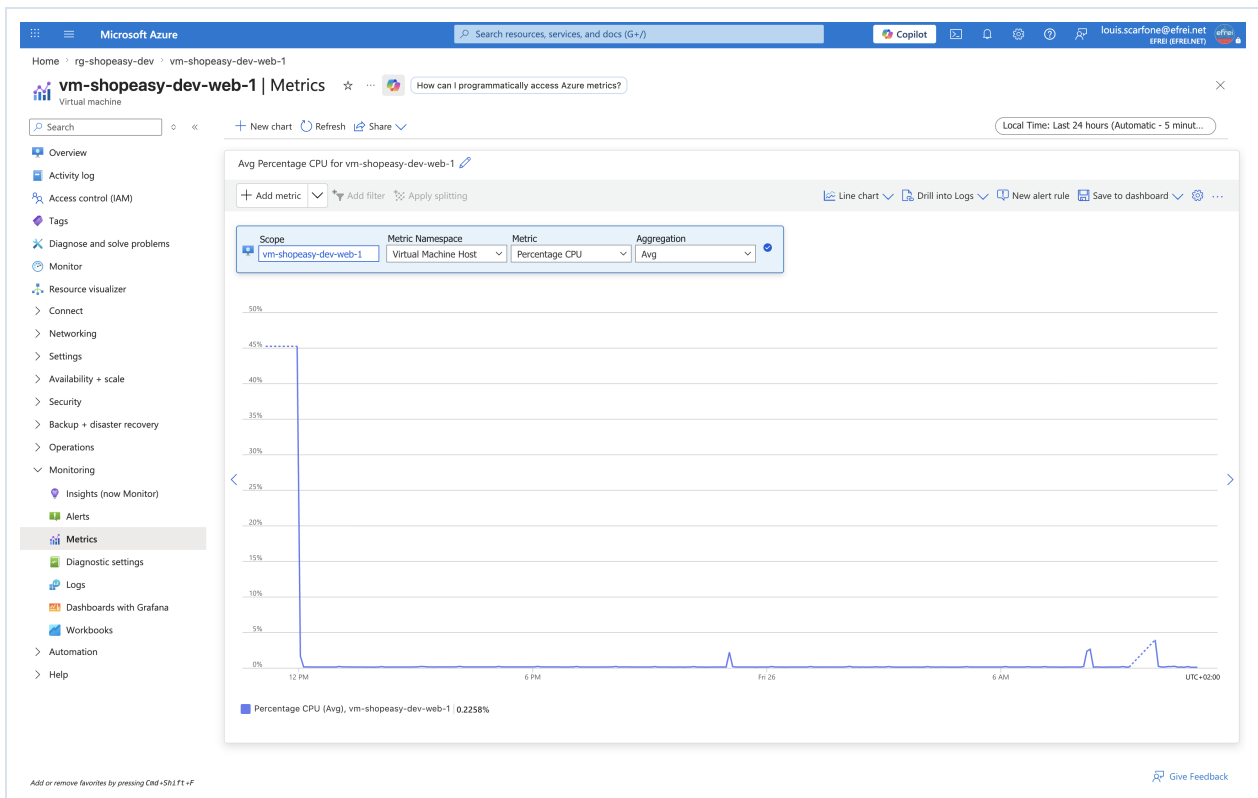
The screenshot shows the 'Alert rules' page in the Azure portal for the resource group 'rg-shopeasy-dev'. The selected alert rule is 'alert-cpu-high-vm-shopeasy-dev-web-1'. The rule is currently 'Enabled' and has a severity of '3 - Informational'. The description is 'Alerte CPU elevee sur VM web ShopEasy'. The scope is set to the virtual machine 'vm-shopeasy-dev-web-1' under the 'Azure for Students' subscription. The conditions are configured with a single rule: 'Percentage CPU > 80', which is monitored 1 time series and has an estimated monthly cost of \$0.10.

Alert rule configuration details:

- Resource group:** rg-shopeasy-dev
- Location:** Global
- Subscription:** Azure for Students
- Subscription ID:** e67b15f9-7cab-4ad0-bc2e-d6e0c0c0e5e4
- Tags:** Add tags
- Severity:** 3 - Informational
- Description:** Alerte CPU elevee sur VM web ShopEasy
- Scope:** vm-shopeasy-dev-web-1 (Hierarchy: Azure for Students > rg-shopeasy-dev)
- Conditions:**

| Name                | Time series monitored | Estimated monthly cost |
|---------------------|-----------------------|------------------------|
| Percentage CPU > 80 | 1                     | \$0.10                 |

Règle d'alerte (Portal → Monitor → Alerts → Alert rules) : `alert-cpu-high-vm-shopeasy-dev-web-1`, Enabled, sévérité 3, condition `Percentage CPU > 80`.



Graphique de la métrique `Percentage CPU` (Portal → Virtual machines → ... → Metrics). Le graphique reste très bas (VM au

repos).

### ✓ État après l'atelier

- Métriques de la VM identifiées (CPU, réseau, disque, crédits CPU burstable) ; CPU lu (baseline 0,12–0,62 %).
- Alerte alert-cpu-high-vm-shopeasy-dev-web-1 créée et active : avg Percentage CPU > 80, fenêtre 5 min, fréquence 1 min, sévérité 3.
- Seuil justifié ; deux alertes complémentaires proposées (disponibilité LB, saturation disque/mémoire).
- Limites des alertes mal calibrées documentées (fatigue d'alerte vs détection tardive).
- **Prêt pour l'Atelier 9 — Écrire un script de contrôle de santé.**

## 9 Atelier 9 — Écrire un script de contrôle de santé

🎯 **Objectif** — créer un script qui vérifie rapidement si l'environnement est dans un état acceptable.

📦 **Livrable attendu** — script `healthcheck.sh` final, preuve d'exécution et commentaires sur les vérifications ajoutées.

### 9.1 Le script `scripts/healthcheck.sh`

Huit contrôles (4 de base + 5 améliorations), code de sortie 0 (conforme) ou 1 (avertissement).

`>_ scripts/healthcheck.sh`

```
#!/usr/bin/env bash
set -euo pipefail
RG="${1:-rg-shopeasy-dev}"; CONTAINER="${2:-operations}"; STATUS=0
ok() { echo " [OK] $1"; }
warn() { echo " [WARN] $1"; STATUS=1; }
ko() { echo " [KO] $1"; STATUS=1; }

# 1. Groupe de ressources (bloquant)
az group show --name "$RG" >/dev/null 2>&1 && ok "groupe '$RG' present" \
|| { ko "RG introuvable"; exit 1; }

# 2. Au moins une VM
NB_VM=$(az vm list -g "$RG" --query "length(@)" -o tsv)
[[ "$NB_VM" -ge 1 ]] && ok "$NB_VM VM(s) presente(s)" || warn "aucune VM"

# 3. Tag Owner sur le groupe
OWNER=$(az group show --name "$RG" --query "tags.Owner" -o tsv)
[[ -n "$OWNER" ]] && ok "tag Owner present ($OWNER)" || warn "tag Owner absent"

# 4. Ressources sans tag Application
UNTAGGED=$(az resource list -g "$RG" --query "length([?tags.Application==null])" -o tsv)
[[ "$UNTAGGED" -eq 0 ]] && ok "toutes taguees Application" || warn "$UNTAGGED sans tag Application"

# 5. Alertes Azure Monitor
ALERTS=$(az monitor metrics alert list -g "$RG" --query "length(@)" -o tsv)
[[ "$ALERTS" -ge 1 ]] && ok "$ALERTS alerte(s)" || warn "aucune alerte configuree"
```

```
# 6. Storage Account (recupere dynamiquement)
STORAGE=$(az storage account list -g "$RG" --query "[0].name" -o tsv 2>/dev/null || true)
[[ -n "$STORAGE" ]] && ok "Storage present ($STORAGE)" || warn "aucun Storage Account"

# 7. Conteneur operations
az storage container show --account-name "$STORAGE" --name "$CONTAINER" \
  --auth-mode login >/dev/null 2>&1 && ok "conteneur '$CONTAINER' present" \
  || warn "conteneur '$CONTAINER' absent"

# 8. Regle NSG HTTP/HTTPS
NB_HTTP=$(az network nsg list -g "$RG" \
  --query "length([].securityRules[?direction=='Inbound' && access=='Allow' && \
    (destinationPortRange=='80' || destinationPortRange=='443')])" -o tsv)
[[ "$NB_HTTP" -ge 1 ]] && ok "$NB_HTTP regle(s) NSG HTTP/HTTPS" || warn "aucune regle HTTP/HTTPS"

exit $STATUS
```

## 9.2 Preuve d'exécution

>\_ console

```
1. Groupe de ressources      [OK]   groupe 'rg-shopeasy-dev' present
2. Machines virtuelles      [OK]   2 VM(s) presente(s)
3. Gouvernance - Owner      [OK]   tag Owner present (cloudops-shopeasy)
4. Gouvernance - Application [WARN] 11 ressource(s) sans tag Application
5. Supervision - alertes    [OK]   1 alerte(s) configuree(s)
6. Stockage - compte        [OK]   Storage present (shopeasydevdocs350wnq)
7. Stockage - operations    [OK]   conteneur 'operations' present
8. Réseau - NSG HTTP/HTTPS [OK]   1 regle(s) NSG autorisant HTTP/HTTPS
RESULTAT : ATTENTION - points a verifier (voir [WARN])    >>> Code de sortie : 1
```

**7 contrôles sur 8 au vert.** Le seul avertissement — **11 ressources sans tag Application** — est cohérent avec l'Atelier 3. Le code 1 permet d'intégrer le script en **CI/cron** (un code  $\neq 0$  déclenche une notification).

## 9.3 Vérifications ajoutées

| Contrôle ajouté      | Pourquoi                                                         |
|----------------------|------------------------------------------------------------------|
| Au moins une VM      | Un groupe sans VM = service web absent.                          |
| Tag Owner            | Sans responsable identifié, pas de gouvernance (qui contacter?). |
| Storage Account      | Sans stockage, impossible d'archiver les rapports.               |
| Conteneur operations | Valide l'espace de dépôt des rapports.                           |
| Règle NSG HTTP/HTTPS | Sans ouverture 80/443, le site est injoignable.                  |

**✓ État après l'atelier**

- Script scripts/healthcheck.sh final : **8 contrôles**, code de sortie 0/1.
- Exécution réelle : **7 OK / 1 WARN** (11 ressources sans tag Application), code 1 → intégrable en CI/cron.
- Contrôles de gouvernance, stockage et réseau ajoutés et commentés; Storage récupéré dynamiquement.
- **Prêt pour l'Atelier 10 — Automatiser avec Python et Azure SDK.**

## 10 Atelier 10 — Automatiser avec Python et Azure SDK

🎯 **Objectif** — découvrir le SDK Azure pour Python afin de produire un inventaire programmable.

📦 **Livrable attendu** — script `inventory.py` et fichier CSV généré, lisible dans Excel ou LibreOffice Calc.

### 10.1 Installation des dépendances

>\_ bash

```
python3 -m venv .venv && source .venv/bin/activate
pip install azure-identity azure-mgmt-resource azure-mgmt-compute
```

>\_ console

```
azure-identity 1.25.3 | azure-mgmt-resource 26.0.0 | azure-mgmt-compute 38.1.0
```

### 10.2 Le script python/inventory.py

>\_ python/inventory.py

```
#!/usr/bin/env python3
"""inventory.py - Inventaire programmable de ShopEasy via l'Azure SDK -> CSV."""
import os, csv
from azure.identity import DefaultAzureCredential
from azure.mgmt.compute import ComputeManagementClient
try:
    from azure.mgmt.resource import ResourceManagementClient # SDK < 26
except ImportError:
    from azure.mgmt.resource.resources import ResourceManagementClient # SDK >= 26

subscription_id = os.environ.get("AZURE_SUBSCRIPTION_ID")
resource_group = os.environ.get("AZURE_RESOURCE_GROUP", "rg-shopeasy-dev")
if not subscription_id:
    raise SystemExit("Variable AZURE_SUBSCRIPTION_ID manquante")
```

```

credential = DefaultAzureCredential()
resource_client = ResourceManagementClient(credential, subscription_id)
compute_client = ComputeManagementClient(credential, subscription_id)

def deduce_role(name, res_type):
    t = res_type.lower(); n = name.lower()
    if "virtualmachines" in t: return "Serveur web (Ubuntu + Nginx)"
    if "loadbalancers" in t: return "Repartiteur de charge (point d'entree HTTP)"
    if "publicipaddresses" in t: return "IP publique du LB" if "lb" in n else "IP publique VM"
    if "storageaccounts" in t: return "Stockage documentaire prive"
    if "insights" in t: return "Alerte de supervision (Azure Monitor)"
    return "Autre" # (+ VNet, NSG, NIC, disks)

def fmt_tags(tags):
    return ";".join(f"{k}={v}" for k, v in sorted(tags.items())) if tags else "(aucun)"

vm_sizes = {vm.name.lower(): vm.hardware_profile.vm_size
             for vm in compute_client.virtual_machines.list(resource_group)}
rows = []
for res in resource_client.resources.list_by_resource_group(resource_group):
    rows.append({"Nom": res.name, "Type": res.type, "Region": res.location,
                "Taille": vm_sizes.get(res.name.lower(), ""),
                "Tags": fmt_tags(res.tags), "Role": deduce_role(res.name, res.type)})
rows.sort(key=lambda r: (r["Type"], r["Nom"]))
with open("exports/inventory.csv", "w", newline="", encoding="utf-8-sig") as f:
    w = csv.DictWriter(f, fieldnames=["Nom", "Type", "Region", "Taille", "Tags", "Role"])
    w.writeheader(); w.writerows(rows)
print(f"Ressources inventoriees: {len(rows)} | VM: {len(vm_sizes)} | CSV genere")

```

### ⚠ Adaptation au SDK récent

Le code du sujet (`from azure.mgmt.resource import ResourceManagementClient`) **échoue** avec `azure-mgmt-resource 26.0.0` : le client a été déplacé dans le sous-module `azure.mgmt.resource.resources`. Le script gère les deux cas via un `try/except ImportError` (rétrocompatible).

## 10.3 Exécution et CSV généré

`DefaultAzureCredential` réutilise la session `az login` (via `AzureCliCredential`), sans secret.

### > console

```

Inventaire du groupe : rg-shopeasy-dev
Ressources inventoriees: 14
Machines virtuelles : 2
CSV genere : exports/inventory.csv

```

### > exports/inventory.csv

```

Nom,Type,Region,Taille,Tags,Role
vm-shopeasy-dev-web-1,Microsoft.Compute/virtualMachines,swedencentral,Standard_B2ats_v2,
  Application=shopeasy;Environment=dev;Owner=cloudops-shopeasy;Role=web,Serveur web (Ubuntu + Nginx)
alert-cpu-high-vm-shopeasy-dev-web-1,Microsoft.Insights/metricalerts,global,,(aucun),

```



Alerte de supervision (Azure Monitor)  
lb-shopeasy-dev-web,Microsoft.Network/loadBalancers,swedencentral,,...,Repartiteur de charge

### 💡 14 ressources via le SDK, 13 via la CLI

Le SDK Python retourne l'**alerte métrique** `Microsoft.Insights/metricAlerts` (créée à l'Atelier 8), que `az resource list` **n'inclut pas** dans son décompte. Ce n'est pas une incohérence mais une **différence de comportement** documentée entre les deux outils.

## 10.4 Bash ou Python ?

| Besoin                                                                   | Outil adapté                                                            |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Enchaîner des commandes, contrôles rapides, exports bruts                | <b>Bash</b> ( <code>inventory.sh</code> , <code>healthcheck.sh</code> ) |
| Enrichir / croiser des données, format structuré (CSV), intégrer une API | <b>Python + SDK</b> ( <code>inventory.py</code> )                       |

Ici, le **croisement** ressources ↔ tailles de VM et la **déduction de rôle** illustrent le moment où Python devient préférable : le besoin n'est plus « lister » mais « **structurer et enrichir** ».

## 10.5 Capture

|    | A                                                              | B                                       | C             | D                 | E                                                                                                               | F                                           |
|----|----------------------------------------------------------------|-----------------------------------------|---------------|-------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------|
|    | Nom                                                            | Type                                    | Region        | Taille            | Tags                                                                                                            | Role                                        |
| 1  | vm-shopeasy-dev-web-1_OsDisk_1_0a80e1b0d80471889b4aed939ad2a   | Microsoft.Compute/disks                 | swedencentral |                   | cost_center=cloud-training.environment=dev.managed_by=terraform.owner=formation=project=shopeasy                | Disque OS manage                            |
| 2  | vm-shopeasy-dev-web-2_OsDisk_1_6fe73e0bf0a8402a8c3cb790c8a4cbe | Microsoft.Compute/disks                 | swedencentral |                   | cost_center=cloud-training.environment=dev.managed_by=terraform.owner=formation=project=shopeasy                | Disque OS manage                            |
| 3  | vm-shopeasy-dev-web-1                                          | Microsoft.Compute/VirtualMachines       | swedencentral | Standard_B2ats_v2 | Application=shopeasy;CostCenter=cloud-training.Environment=dev.ManagedBy=mixed.Owner=cloudops-shopeasy;Role=web | Serveur web (Ubuntu + Nginx)                |
| 4  | vm-shopeasy-dev-web-2                                          | Microsoft.Compute/VirtualMachines       | swedencentral | Standard_B2ats_v2 | Application=shopeasy;CostCenter=cloud-training.Environment=dev.ManagedBy=mixed.Owner=cloudops-shopeasy;Role=web | Serveur web (Ubuntu + Nginx)                |
| 5  | alert-cpu-high-vm-shopeasy-dev-web-1                           | Microsoft.Insights/metricAlerts         | global        |                   | (aucun)                                                                                                         | Alerte de supervision (Azure Monitor)       |
| 6  | lb-shopeasy-dev-web                                            | Microsoft.Network/loadBalancers         | swedencentral |                   | cost_center=cloud-training.environment=dev.managed_by=terraform.owner=formation=project=shopeasy                | Repartiteur de charge (point d'entrée HTTP) |
| 7  | nic-shopeasy-dev-web-1                                         | Microsoft.Network/networkInterfaces     | swedencentral |                   | cost_center=cloud-training.environment=dev.managed_by=terraform.owner=formation=project=shopeasy                | Carte réseau de VM                          |
| 8  | nic-shopeasy-dev-web-2                                         | Microsoft.Network/networkInterfaces     | swedencentral |                   | cost_center=cloud-training.environment=dev.managed_by=terraform.owner=formation=project=shopeasy                | Carte réseau de VM                          |
| 9  | nsg-shopeasy-dev-web                                           | Microsoft.Network/networkSecurityGroups | swedencentral |                   | cost_center=cloud-training.environment=dev.managed_by=terraform.owner=formation=project=shopeasy                | Pare-feu réseau (NSG)                       |
| 10 | pip-shopeasy-dev-lb                                            | Microsoft.Network/publicIPAddresses     | swedencentral |                   | cost_center=cloud-training.environment=dev.managed_by=terraform.owner=formation=project=shopeasy                | IP publique du Load Balancer                |
| 11 | pip-shopeasy-dev-web-1                                         | Microsoft.Network/publicIPAddresses     | swedencentral |                   | cost_center=cloud-training.environment=dev.managed_by=terraform.owner=formation=project=shopeasy                | IP publique d'administration VM             |
| 12 | pip-shopeasy-dev-web-2                                         | Microsoft.Network/publicIPAddresses     | swedencentral |                   | cost_center=cloud-training.environment=dev.managed_by=terraform.owner=formation=project=shopeasy                | IP publique d'administration VM             |
| 13 | vnet-shopeasy-dev                                              | Microsoft.Network/virtualNetworks       | swedencentral |                   | cost_center=cloud-training.environment=dev.managed_by=terraform.owner=formation=project=shopeasy                | Réseau privé (segmentation)                 |
| 14 | shopeasydevdocs350winq                                         | Microsoft.Storage/storageAccounts       | swedencentral |                   | cost_center=cloud-training.environment=dev.managed_by=terraform.owner=formation=project=shopeasy                | Stockage documentaire privé                 |

Le fichier `exports/inventory.csv` ouvert dans un tableur : les colonnes `Nom`, `Type`, `Region`, `Taille`, `Tags`, `Role` s'affichent correctement (encodage `utf-8-sig`).

### ✓ État après l'atelier

- Environnement Python isolé (`.venv`) ; SDK Azure installé (`identity`, `mgmt-resource`, `mgmt-compute`).
- Script `python/inventory.py` fonctionnel : authentification `az login`, **CSV enrichi** (rôle + taille).

- Adaptation au SDK récent gérée (ResourceManagementClient déplacé en v26, import rétrocompatible).
- exports/inventory.csv (14 ressources) lisible dans un tableur ; classe l'alerte de supervision.
- **Prêt pour l'Atelier 11 — Analyse FinOps d'exploitation.**

## 11 Atelier 11 — Analyse FinOps d'exploitation

🎯 **Objectif** — identifier les actions simples permettant de limiter les coûts d'un environnement de développement.

📦 **Livrable attendu** — tableau FinOps complété et trois recommandations courtes pour la DSI.

### 11.1 Ressources les plus coûteuses

Coûts calculés à partir des **prix réels** de l'API Azure Retail Prices (swedencentral, USD, base 730 h/mois).

| Poste                     | Détail                 | Coût/mois  | Part  |
|---------------------------|------------------------|------------|-------|
| Load Balancer Standard    | forfait règles + data  | ≈ 18,25 \$ | ~39 % |
| Compute — 2 VM B2ats_v2   | 2 × 0,00972 \$/h × 730 | ≈ 14,19 \$ | ~30 % |
| 3 IP publiques Standard   | 3 × 0,005 \$/h × 730   | ≈ 10,95 \$ | ~23 % |
| 2 disques OS Standard_LRS | ~30 Go HDD             | ≈ 2,60 \$  | ~6 %  |
| Storage Account LRS       | usage faible           | < 1 \$     | ~2 %  |
| <b>Total</b>              |                        | ≈ 47 \$    | 100 % |

Les trois premiers postes représentent ~92 % du coût. Le **Load Balancer** et les **IP** sont facturés 24h/24, **indépendamment du trafic** ; seul le **compute** est élastique.

#### 💡 Aucune ressource orpheline

`az disk list --query "[?managedBy==null]"` et `az network public-ip list --query "[?ipConfiguration==null]"` → **aucun disque orphelin, aucune IP non associée**. L'environnement est propre, sans coût « invisible ».

## 11.2 Tableau FinOps

| Action                           | Effet                       | Risque / limite          | Application à ShopEasy                                    |
|----------------------------------|-----------------------------|--------------------------|-----------------------------------------------------------|
| Désallouer les VM hors usage     | Réduction du coût compute   | Indisponibilité          | vm-power.sh : VM2 désallouée par défaut, VM1 hors heures. |
| Réduire la taille des VM         | Réduction du coût mensuel   | Performance plus faible  | B2ats_v2 déjà petite; surveiller les crédits CPU.         |
| Supprimer les disques inutilisés | Réduction du stockage       | Perte de donnée utile    | Aucun disque orphelin; contrôle automatisé.               |
| Standardiser les tags            | Meilleure analyse des coûts | Discipline nécessaire    | Étendre aux 11 ressources restantes.                      |
| Définir un budget                | Alerte en cas de dérive     | Ne bloque pas la dépense | Budget 50 \$/mois, alertes 80 / 100 %.                    |

## 11.3 Stratégie d'arrêt/démarrage & budget

- **Désallocation hors heures ouvrées** (vm-power.sh deallocate le soir, start le matin) : une VM utilisée ~50 h/semaine économise ~70 % de son compute.
- **Automatisation** : Azure Automation runbook, Logic App ou cron; tag AutoShutdown.
- **VM secondaire web-2 (HA)** : **désallouée par défaut** en dev.
- **Levier maximal** : terraform destroy entre les séances → coût  $\approx 0$ , recréable par apply.
- **Budget** : 50 \$/mois sur le RG, alertes 80 % (40 \$) et 100 % (50 \$) — un budget alerte mais ne bloque pas.

## 11.4 Trois recommandations FinOps pour la DSI

| # | Recommandation                                                                  | Gain                               |
|---|---------------------------------------------------------------------------------|------------------------------------|
| 1 | <b>Désallocation automatisée</b> des VM dev + destruction entre séances         | −60 à −70 % compute                |
| 2 | <b>Supprimer les IP publiques des VM</b> , administrer via <b>Azure Bastion</b> | $\approx 7$ \$/mois + sécurité     |
| 3 | <b>Imposer les tags par Azure Policy</b> + budget mensuel avec alertes          | gouvernance, détection des dérives |

### ✓ État après l'atelier

- Coût réel chiffré (prix Azure Retail Prices) :  $\approx 47$  \$/mois en 24/7, dont ~92 % sur LB + compute + IP.
- Environnement **propre** : aucun disque orphelin, aucune IP non associée.

- Tableau FinOps complété (5 actions × effet × risque × application ShopEasy).
- Stratégie d'arrêt, politique de tags, budget (50 \$) et **3 recommandations DSI** prêtes.
- **Prêt pour l'Atelier 12 — Rapport d'exploitation ShopEasy.**

## 12 Atelier 12 — Rapport d'exploitation ShopEasy

**Objet** — exploitation et automatisation de l'environnement Azure de ShopEasy

**Périmètre** — groupe de ressources rg-shopeasy-dev (swedencentral), abonnement Azure for Students

**Auteurs (binôme)** — Louis SCARFONE & Maxence BOURRAGUÉ

**Double lecture** — l'équipe technique (commandes rejouables) et la DSI (risques, coûts, priorités)

### 12.1 Contexte et périmètre

ShopEasy a conçu son architecture au TP1 (déploiement manuel) puis l'a industrialisée au TP2 (Terraform). Le TP3 ouvre la phase d'**exploitation** : administrer, superviser, sécuriser et optimiser avec **Azure CLI, Bash et Python**. L'environnement a été **redéployé à l'identique** depuis le code Terraform du TP2 (21 objets), puis exploité sans repasser par le portail.

### 12.2 Inventaire des ressources

Inventaire produit en CLI, exporté en JSON/TSV, enrichi en CSV par le SDK Python. **13 ressources d'infrastructure** (décompte `az resource list`) + **1 alerte de supervision** retournée par le SDK, soit **14 objets ARM** : 2 VM web, 2 disques, 3 IP publiques, 2 NIC, 1 VNet, 1 NSG, 1 Load Balancer, 1 Storage Account, 1 alerte métrique. **Statut : conforme.**

### 12.3 Tags et gouvernance

Convention d'exploitation **normalisée** (Application, Environment, Owner, CostCenter, ManagedBy) appliquée au RG et aux 2 VM, en remplacement des tags techniques Terraform. **11 ressources** conservent encore les tags `snake_case` — écart détecté automatiquement par `inventory.sh` et `healthcheck.sh`. **Statut : partiel.**

### 12.4 État des VM et actions réalisées

Administration complète en CLI, sans portail ni SSH direct : `run-command` (Nginx active, HTTP 200), `restart` sur web-1, `deallocate` puis `start` sur web-2. Distinction `stop` / `deallocate` maîtrisée. **Statut : conforme.**

## 12.5 Stockage opérationnel

Storage Account du TP2 **réutilisé et sécurisé** : accès public **désactivé** (AllowBlobPublicAccess=False), TLS 1.2, HTTPS-only ; conteneur **privé operations** ; authentification par **RBAC data-plane** (sans clé partagée) ; accès anonyme **prouvé refusé** (HTTP 409). **Statut : conforme.**

## 12.6 Monitoring et alertes

Métriques identifiées (CPU, réseau, disque, crédits CPU) ; **alerte active** alert-cpu-high-vm-shopeasy-dev-web-1 (avg Percentage CPU > 80, fenêtre 5 min, sévérité 3) ; deux alertes complémentaires **proposées** (disponibilité LB, saturation disque). **Statut : partiel.**

## 12.7 Scripts produits

| Script         | Rôle                                                                              |
|----------------|-----------------------------------------------------------------------------------|
| inventory.sh   | Inventaire + synthèse (compteurs, ressources sans tag, avertissement VM running). |
| vm-power.sh    | Start/stop/deallocate/status, garde-fou prod, confirmation, journal.              |
| healthcheck.sh | 8 contrôles de santé, code de sortie 0/1 (CI/cron).                               |
| inventory.py   | Inventaire SDK → CSV (rôle déduit, taille VM, utf-8-sig).                         |
| variables.sh   | Variables centralisées, aucun secret en dur.                                      |

Tous commentés, paramétrables, relançables, sans action destructive non confirmée. **Statut : conforme.**

## 12.8 Analyse sécurité

**Contrôles en place** : SSH restreint à l'IP admin (/32) ; HTTP limité au port 80 ; stockage privé + RBAC (moindre privilège) ; aucun secret versionné ; administration sans SSH direct (run-command). **Risques résiduels** : IP publiques directes sur les VM, absence d'Azure Bastion, journalisation/diagnostic non centralisée. **Statut : conforme** sur l'essentiel, renforcements identifiés.

## 12.9 Analyse FinOps

Coût réel ≈ **47 \$/mois en 24/7** : Load Balancer (~39 %), compute (~30 %), IP publiques (~23 %). Le LB et les IP facturent même à trafic nul ; seul le compute est élastique. Aucune ressource orpheline. **Statut : partiel** — coût chiffré et leviers identifiés, budget et désallocation auto à appliquer.

## 12.10 Recommandations d'amélioration (priorisées)

| # | Recommandation                                        | Gain                   | Priorité |
|---|-------------------------------------------------------|------------------------|----------|
| 1 | Désallocation automatisée + destruction entre séances | −60 à −70 % compute    | Haute    |
| 2 | Supprimer les IP publiques des VM (Azure Bastion)     | ≈ 7 \$/mois + sécurité | Haute    |
| 3 | Généraliser les tags + Azure Policy                   | gouvernance            | Moyenne  |
| 4 | Budget (50 \$/mois) avec alertes 80/100 %             | détection des dérives  | Moyenne  |
| 5 | Compléter les alertes (LB, disque) + action group     | détection des pannes   | Moyenne  |
| 6 | Diagnostic settings / Activity Log centralisés        | auditabilité           | Basse    |

### Tableau de synthèse

| Domaine    | Statut     | Commentaire                                                                           |
|------------|------------|---------------------------------------------------------------------------------------|
| Inventaire | ● Conforme | Inventaire CLI complet (14 objets), exports JSON/TSV/CSV, rejouable.                  |
| Tags       | ● Partiel  | RG + 2 VM normalisés ; 11 ressources encore en snake_case.                            |
| VM         | ● Conforme | États, run-command, start/stop/deallocate maîtrisés.                                  |
| Stockage   | ● Conforme | Conteneur privé, accès public désactivé, RBAC, accès anonyme refusé (409).            |
| Monitoring | ● Partiel  | Alerte CPU active ; alertes disponibilité/disque et action group à créer.             |
| Sécurité   | ● Conforme | SSH restreint, stockage privé, RBAC, aucun secret versionné ; Bastion recommandé.     |
| FinOps     | ● Partiel  | Coût chiffré (~47 \$/mois), leviers identifiés ; budget et désallocation à appliquer. |

### Conclusion

L'environnement ShopEasy est désormais **exploité de manière outillée et reproductible** : inventaire automatisé, administration scriptée et sécurisée, stockage verrouillé, supervision initiale et analyse de coûts chiffrée. Les fondamentaux (inventaire, VM, stockage, sécurité) sont **conformes** ; les axes **tags**, **monitoring** et **FinOps** sont engagés et **partiellement** appliqués, avec une feuille de route claire. L'ensemble constitue un **mini-runbook** permettant à une autre personne de reprendre l'administration **sans dépendre du portail Azure**.

## ✓ Fin du TP3

- Rapport d'exploitation complet (10 sections + tableau de synthèse), déposé dans le conteneur `operations/rapports/`.
- Bilan : **4 domaines conformes** (inventaire, VM, stockage, sécurité), **3 partiels** (tags, monitoring, FinOps) avec recommandations.
- Mini-kit d'exploitation livré : `variables.sh`, 3 scripts Bash, `inventory.py`, `exports`, `logs` et `rapport`.

## 13 Quiz de validation

1. **Différence entre `az vm stop` et `az vm deallocate` ?**  
`stop` arrête l'OS mais la VM reste **allouée** (compute facturé) ; `deallocate` arrête **et libère** le calcul (*Stopped (deallocated)*, compute non facturé). En dev, privilégier `deallocate`.
2. **Pourquoi Azure CLI plutôt que le portail pour une tâche répétitive ?**  
Une commande est **rejouable**, documentable, intégrable à un script et exécutable en pipeline. Le portail est manuel, peu reproductible et difficile à tracer.
3. **À quoi sert l'option `--query` ?**  
À filtrer et transformer les sorties JSON avec le langage **JMESPath** (sélectionner des champs, filtrer, renommer des colonnes).
4. **Quel format de sortie pour réutiliser un résultat dans un script Bash ?**  
Le **TSV** (`-o tsv`) : sans guillemets ni en-tête, il s'extrait directement dans une variable Bash.
5. **Pourquoi les tags sont-ils importants pour le FinOps ?**  
Ils permettent de **ventiler la facture** (application, environnement, CostCenter), d'identifier le responsable (owner) et d'automatiser des politiques (arrêt des dev).
6. **Qu'est-ce qu'un runbook d'exploitation ?**  
Une **procédure documentée** décrivant comment réaliser une action de façon fiable (démarrer/arrêter, vérifier la santé, réagir à une alerte). Les scripts du TP3 forment un mini-runbook.
7. **Pourquoi éviter d'ouvrir SSH à tout Internet ?**  
Exposer le port 22 à `0.0.0.0/0` l'expose aux **scans** et attaques par **force brute**. On restreint à l'IP admin (`/32`).
8. **Quelle commande liste les ressources d'un groupe ?**  
`az resource list --resource-group <rg>`.
9. **Quel service suit les métriques et crée des alertes ?**  
**Azure Monitor** (métriques, logs, alertes, action groups).
10. **Différence entre métrique et log ?**  
Une **métrique** est une valeur **numérique** dans le temps (CPU %, débit) ; un **log** est un **événement détaillé** (texte structuré). La métrique mesure, le log raconte.
11. **Pourquoi versionner les scripts d'exploitation ?**  
Pour **tracer** les modifications, revenir en arrière, collaborer (revue de code) et garantir la même version fiable. Un script d'exploitation est du code.
12. **Rôle de `DefaultAzureCredential` dans un script Python Azure ?**  
Une **chaîne d'authentification** qui tente plusieurs sources (env, identité managée, `az login`) pour obtenir un token Azure AD **sans secret en dur**.
13. **Pourquoi tester un script sur un environnement non productif ?**

- Un script peut **arrêter, modifier ou supprimer** des ressources. Le tester en dev évite d'impacter la production en cas de bug ou de mauvaise variable.
14. **Deux risques d'un script d'administration mal sécurisé ?**  
**Action destructive sans confirmation** (arrêt/suppression accidentelle) ; **secrets en clair** dans le code (clé, mot de passe) exposés en cas de fuite.
  15. **Quelle information doit figurer dans un rapport d'exploitation ?**  
Inventaire, état des VM/services, contrôle des tags/gouvernance, sécurité, supervision, **analyse des coûts**, risques et **recommandations priorisées** — pour l'équipe technique et la DSI.
  16. **Pourquoi un budget ne remplace-t-il pas une gouvernance des ressources ?**  
Un budget **alerte sur un montant** mais ne dit pas quelles ressources, qui en est responsable, ni lesquelles arrêter. Sans tags ni organisation, on voit la dérive sans pouvoir agir ; il ne bloque pas la dépense.
  17. **Deux actions simples pour réduire les coûts d'un environnement de dev ?**  
**Désallouer les VM** hors usage (et détruire l'environnement entre séances) ; **supprimer les ressources orphelines / IP inutilisées** (ou réduire la taille des VM).
  18. **Pourquoi journaliser les actions d'un script ?**  
Pour **tracer** qui a fait quoi et quand (audit), diagnostiquer en cas d'incident et prouver les contrôles. `vm-power.sh` écrit dans `logs/vm-power.log`.
  19. **Différence entre IaC et script d'exploitation ?**  
L'**IaC** (Terraform) décrit l'**état cible** de façon déclarative (construit l'infra). Un **script d'exploitation** réalise des **actions ponctuelles** sur l'existant de façon impérative (administre l'infra). L'un construit, l'autre exploite.
  20. **Citer trois contrôles d'un healthcheck.sh.**  
Existence du **groupe de ressources** (bloquant) ; présence et **état des VM** ; **ressources sans tag** obligatoire (Application/Owner). (*Également : alerte Azure Monitor, Storage + conteneur operations, règle NSG HTTP/HTTPS.*)

## 🚩 Fin du dossier de rendu — TP3 Administration & Automatisation Azure

ShopEasy — Louis SCARFONE & Maxence BOURRAGUÉ — Bloc 4 · Cloud Computing